

Department of Electrical and Computer Engineering
COMSATS University Islamabad, Attock Campus

Lab Rubrics Form	
Lab Name	Iterative Statements (Loops) in C Programming
Lab Number	06
Submitted by:	
Student Names	Registration #
Sumaira Safeer	FA22-BCE-019

Rubrics		Marks		
		1	2	3
PLO-5 Modern Tools Usage	Ability to execute the experiment utilizing hardware, software tools, or a combination of both.			
PLO-10 Communication Skills	Present lab activities effectively through detailed lab reports, oral examinations, and engaging presentations.			

Lab 06: Iterative Statements (Loops) in C Programming

Objective

To understand and implement iterative statements (while and for loops) in C programming for solving repetitive computational problems such as series generation, factorial calculation, number conversion, and mathematical operations.

Software Requirements

- Code::Blocks IDE / Dev-C++
- GCC Compiler.
 - Windows Operating System.

Experiments Performed

In-Lab Tasks

1. Print even numbers from 2 to 100 using a while loop.
2. Calculate the power of a number using repeated multiplication.
3. Display Armstrong numbers between 1 and 500.

Post-Lab Tasks

1. Calculate the factorial of a number using a while loop.
2. Convert a decimal number into its octal equivalent.
3. Calculate overtime pay for ten employees based on overtime hours worked.

Learning Outcomes

After completing this lab, the student will be able to:

- Implement while and for loops in C.
- Solve repetitive mathematical problems using iteration.
- Generate numerical sequences.
- Perform number system conversions.
- Calculate factorials and powers using loops.
- Develop practical applications involving iterative processing.

In-Lab Tasks:

Task1:

Print even numbers from 2 to 100 using a while loop.

Code:

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int number=2;
    while (number<=100)
    {
        printf("%d\t",number);
        number+=2;
    }
    return 0;
}
```

Output:

```
"C:\Users\Hamza Computer\Documents\PF\lab1\kjjk\bin\Debug\kjjk.exe"
2      4      6      8      10     12     14     16     18     20     22     24     26     28     30
32     34     36     38     40     42     44     46     48     50     52     54     56     58
60     62     64     66     68     70     72     74     76     78     80     82     84     86
88     90     92     94     96     98     100
Process returned 0 (0x0) execution time : 0.187 s
Press any key to continue.
```

Task2:

Calculate the power of a number using repeated multiplication.

Code:

```
#include <stdio.h>
#include <stdlib.h>
```

```

int main()
{
    int Base, Exponent;
    int result = 1;
    printf("Enter the Base number: ");
    scanf("%d", &Base);

    printf("Enter the Exponent: ");
    scanf("%d", &Exponent);

    while (Exponent > 0) {
        result *= Base;
        Exponent--;
    }
    printf("Result: %d\n", result);
    return 0;
}

```

Output:

```

Enter the Base number: 564
Enter the Exponent: 3
Result: 179406144

Process returned 0 (0x0)   execution time : 6.745 s
Press any key to continue.
_

```

Task3:

Display Armstrong numbers between 1 and 500.

Code:

```

#include <stdio.h>
#include <stdlib.h>

int main()
{

```

```

int Number, original_Num, remainder, result;

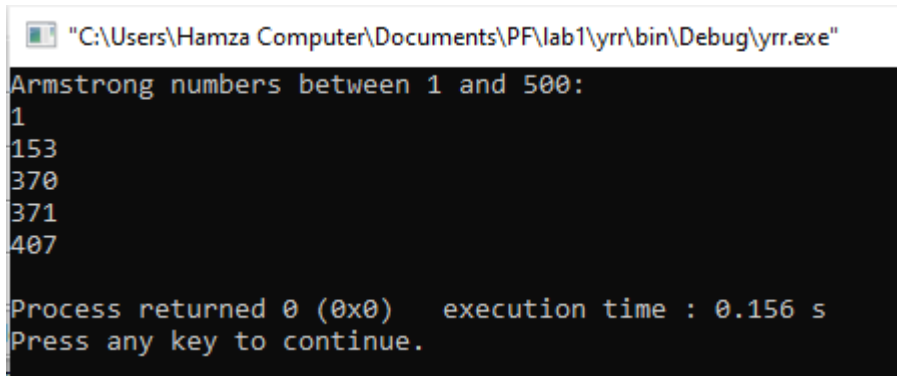
printf("Armstrong numbers between 1 and 500:\n");

for (Number=1; Number<=500; Number++) {
    original_Num= Number;
    result = 0;
    while (original_Num!=0) {

        remainder = original_Num%10;
        result += pow(remainder, 3);
        original_Num /= 10;
    }
    if (result == Number) {
        printf("%d\n", Number);
    } }
return 0;
}

```

Output:



```

"C:\Users\Hamza Computer\Documents\PF\lab1\yrr\bin\Debug\yrr.exe"
Armstrong numbers between 1 and 500:
1
153
370
371
407

Process returned 0 (0x0)   execution time : 0.156 s
Press any key to continue.

```

Post-lab Tasks:

Task1:

Calculate the factorial of a number using a while loop.

Code:

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int number;
    int factorial = 1;
    printf("Enter a number: ");
    scanf("%d", &number);
    int i = 1;
    while (i <= number) {
        factorial *= i;
        i++;
    }
    printf("Factorial of %d is %d\n", number, factorial);
    return 0;
}
```

Task2:

Convert a decimal number into its octal equivalent.

Code:

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int number, original_Num, remainder;
```

```

int octal= 0, Value= 1;

printf("Enter a decimal number:");
scanf("%d", &number);

original_Num = number;
while (number != 0) {
    remainder = number%8;
    octal += remainder * Value;
    number/= 8;
    Value *= 10;
}

printf("Octal equivalent of %d is %d\n", original_Num, octal);
return 0;
}

```

Output:

 "C:\Users\Hamza Computer\Documents\PF\lab1\kjjk\bin\Debug\kjjk.exe"

```

Enter a decimal number:78
Octal equivalent of 78 is 116

Process returned 0 (0x0)   execution time : 4.078 s
Press any key to continue.

```

Task3:

Calculate overtime pay for ten employees based on overtime hours worked.

Code:

```

#include <stdio.h>
#include <stdlib.h>

int main()
{

```

```
int Employee= 1;
int hours_Worked, Overtime_Hours;
int Overtime_Rate=12, Regular_Hour=40;
float Overtime_Pay;

while (Employee <= 10) {
    printf("Enter number of hours worked for Employee %d=", Employee);
    scanf("%d", &hours_Worked);

    if (hours_Worked > Regular_Hour) {
        int overtime_Hours = hours_Worked - Regular_Hour;
        Overtime_Pay = Overtime_Hours * Overtime_Rate;
    } else {
        Overtime_Pay = 0;
    }
    printf("Overtime Pay for Employee %d= Rs.%f \n", Employee, Overtime_Pay);
    Employee++;
}

return 0;
```